

Tema 4: Inmersión y árboles binarios

4.1 La inmersión

Cuando una función se llama a sí misma de forma recursiva, la memoria solicita un nuevo bloque (*frame*) para ejecutar su instancia particular. En el examen no podemos alterar la “firma pública” (si te piden hacer `reverse(string s)`, no puedes añadir argumentos por tu cuenta). La estrategia de la **Inmersión** responde de esta manera: 1. Crear una segunda función auxiliar. 2. Hacer que la función pública pre-cargue esta función de inmersión oculta.

Invertir String (`reverse`)

Necesitamos un acumulador para guardar el *string* girado. Pasando un simple segundo parámetro por inmersión conseguimos llevar el cálculo entre instancias:

```
// 1. Función inmersa (auxiliar)
string reverse__(string s, string invertido) {
    if (s.empty()) return invertido;
    return reverse__(s.substr(1), s[0] + invertido);
}

// 2. Función pública (Interfaz original)
string reverse(string s) {
    return reverse__(s, "");
}
```

Fibonacci en $\mathcal{O}(n)$

La recursión exponencial convencional al calcular Fibonacci repetía llamadas sobre llamadas lastimosamente provocando parálisis matemáticas de $\mathcal{O}(2^n)$. Con una única Inmersión bajamos y solucionamos el coste a lineal $\mathcal{O}(n)$ pasando directamente en el viaje la evolución de los dos últimos números obtenidos.

```
int fibonacci__(int n, int a, int b) {
    if (n == 0) return a;
    return fibonacci__(n - 1, b, a + b);
}

int fibonacci(int n) {
    return fibonacci__(n, 0, 1);
}
```

4.2 El árbol binario (`BinTree<T>`)

Es una estructura de datos estrictamente recursiva: o es un *vacío absoluto*, o tiene un nodo central (*raíz*) asociado como máximo a dos descendientes exactos (*izquierdo* e *derecho*) que, a su vez, son considerados subárboles `BinTree` respectivos en sí mismos.

[Gràfic interactiu disponible a la versió web de l'apunt]

Un árbol BinTree **NO se puede modificar**. Una vez haces el constructor y lo cierras, nunca podrás acceder a licencias como “*coger su rama derecha nativa y borrarla con un delete o set*”. Para alterar datos, se opera **reconstruyendo completamente el mismo árbol como una Instancia Nueva** aprovechando todas las partes antiguas junto con el cambio. (Mira abajo, el apartado 4.3).

4.3 Funciones básicas y de mutación

Vamos a transformar los dos problemas más comunes en la clase BinTree (Saber la altura total `height` o buscar si existe una hoja `búsqueda`):

[Simulació interactiva disponible a la web]

Como el árbol no tiene punteros libres o licencias de asignación de variables nativas como los Vectores positivos, **cualquier operación que “modifique” un árbol** en teoría, en la práctica de C++, lo que hace es reconstruirlo entero creando nuevos nodos por la zona o rama que haya sufrido el cambio. Revisiones inmutables.

4.4 Los recorridos globales

Búsqueda en profundidad (DFS)

Bajar por el túnel hasta el final antes de escanear lateralmente.

- **Preorden:** *Raíz → Izquierdo → Derecho.*

[Simulació interactiva disponible a la web]

- **Inorden:** *Izquierdo → Raíz → Derecho.*

[Simulació interactiva disponible a la web]

- **Postorden:** *Izquierdo → Derecho → Raíz.*

[Simulació interactiva disponible a la web]

Búsqueda en anchura (BFS)

[Simulació interactiva disponible a la web]

4.5 Eficiencia multitarea (`pair<A, B>`)

Si nos piden resolver dos cosas a la vez (ej: *¿Está subequilibrado? ¿Y qué altura tiene?*), lanzar dos funciones de búsqueda separadas provocará un desastre de eficiencia a $\Theta(N^2)$. **La solución:** Buscar en una sola pasada, devolviendo los dos datos a la vez dentro de una tupla `std::pair` ($\Theta(N)$).

A continuación vemos cómo extraer tanto la suma de todos los valores *como* la cantidad de nodos con un solo `std::pair` para averiguar la media global:

[Simulació interactiva disponible a la web]

4.6 Leer y Reconstruir Árboles

En los Jutges del laboratorio recibirás los árboles representados en una línea de texto plano (ej: 10 5 # # 14 # #), donde un # indica “Subárbol Vacío”. Aquí tienes una utilidad rápida auxiliar de conversión:

```
template <typename T>
T leer_valor(string texto) {
    istringstream iss(texto);
    T elem;
    iss >> elem;
    return elem;
}
```

1. Leyendo en formato Preorden (El habitual)

Muy directo: La raíz siempre es la primera en entrar por el cin. Después vienen los de la izquierda, y finalmente los de la derecha.

[Simulació interactiva disponible a la web]

2. Leyendo en formato Postorden (Con Pila stack)

Si no hay más remedio y te lo dan en Postorden, la lectura normal falla porque “la información de la raíz llega en el último segundo a tu teclado”. Tendremos que leerlo todo al revés apilando directamente en un stack hasta resolver el camino entero hacia arriba:

```
<!--“cpp template pro2::BinTree bintree_desde_postorden(istream& in) { stack<pro2::BinTree> S; string
token;

while (in >> token) {
    if (token == "#" || !in) {
        S.push(pro2::BinTree<T>());
    } else {
        T valor = leer_valor<T>(token);

        // Verificación por si la entrada rompe el assert de índice.
        assert(S.size() >= 2);

        // ¡Cuidado al invertir! La Derecha domina la parte superior y recibirá el pop primero
        auto derecha = S.top(); S.pop();
        auto izquierda = S.top(); S.pop();

        // ¡Árbol entero reconstruido hacia arriba!
        S.push(pro2::BinTree<T>(valor, izquierda, derecha));
    }
}
assert(S.size() == 1);
return S.top();
} “
```